

Búsqueda y planeación en los agentes inteligentes

Impartido por: Paulo Daniel Vázquez Mora

Búsqueda y planeación en los agentes inteligentes

Esta presentación ayuda a entender cómo los agentes inteligentes toman decisiones. Además, se introduce el concepto fundamental del espacio de estados como un mapa de posibles situaciones. Por otro lado, se realizará una revisión de los algoritmos de planificación, que van más allá de la simple búsqueda para construir planes o secuencias de acciones.



Introducción

- Recordemos que un **agente inteligente** es una entidad que percibe su entorno y actúa para lograr sus objetivos. El verdadero desafío para un agente no es solo percibir, sino también decidir qué acciones tomar para pasar de un estado actual a un estado deseado.
- Es ahí donde entran en juego la **búsqueda** y la **planificación**. Estos algoritmos son el "cerebro" detrás de la toma de decisiones, permitiendo que un agente navegue en un mundo complejo y logre sus metas de manera autónoma.

Fundamentos: el espacio de estados (1/2)

Para que un agente pueda **tomar decisiones**, necesita un modelo mental de su entorno, conocido como **espacio de estados**. Este espacio es como un mapa o un laberinto donde cada punto (o nodo) es un posible estado del mundo, como una posición específica. Las acciones son los movimientos que conectan un estado con otro. El objetivo es encontrar una secuencia de acciones que lleve al agente del estado inicial al estado objetivo. Comprender este concepto es la base para entender todos los algoritmos que siguen.

Fundamentos: el espacio de estados (2/2)

Ejemplo: Imagina un juego de 15 rompecabezas donde las piezas numeradas del 1 al 15 se pueden deslizar en un tablero de 4x4 con un espacio vacío.

- **Estado:** La disposición actual de los números en el tablero.
- **Estado inicial:** La configuración inicial del rompecabezas.
- **Acciones:** Mover el espacio vacío (arriba, abajo, izquierda, derecha).
- **Estado objetivo:** La configuración final ordenada del 1 al 15.



Categorías de algoritmos de búsqueda (1/2)

Los **algoritmos de búsqueda** se dividen en dos grandes grupos:

Los algoritmos de **búsqueda no informada**

- También llamada búsqueda "**ciega**", no tienen conocimiento previo sobre el problema y exploran el espacio de estados de manera sistemática.

Los algoritmos de **búsqueda informada**

- Utilizan una **función heurística**, una especie de "intuición", para guiar la búsqueda hacia la solución. Son mucho más rápidos, aunque su eficacia depende de la calidad de la heurística.

Conceptos clave para algoritmos de búsqueda

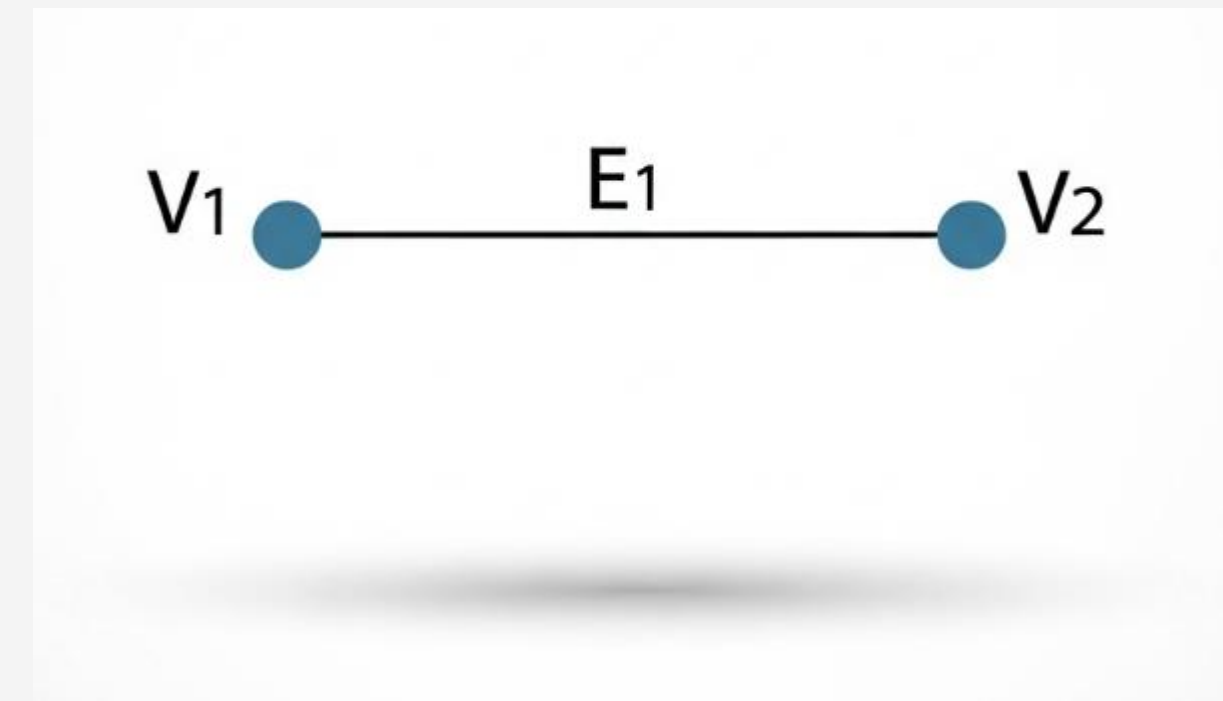
Los algoritmos de búsqueda, como el de búsqueda en profundidad (DFS) y el de búsqueda en anchura (BFS), exploran las conexiones de un grafo para encontrar una ruta o un nodo específico. Para ello, es importante entender los siguientes conceptos:

- **Vecinos (o nodos adyacentes):** Son los vértices que están directamente conectados por una sola arista.
- **Ruta (o camino):** Una secuencia de aristas que conecta una serie de vértices.
- **Conectividad:** Un grafo es conectado si existe una ruta entre cualquier par de vértices. Si no lo es, se dice que es desconectado.
- **Ciclo:** Una ruta que comienza y termina en el mismo vértice.
- **Árbol:** Un tipo especial de grafo no dirigido, conectado y sin ciclos. Esencialmente, es un grafo que no tiene circuitos.

Componentes de un grafo(G)

Un grafo G se define formalmente como un par ordenado (V,E) , donde V es un conjunto de vértices y E es un conjunto de aristas.

- **Vértices (V):** Son los puntos individuales en un grafo. Imagina los vértices como ciudades en un mapa; cada ciudad es un vértice.
- **Aristas (E):** Son las líneas que conectan los vértices. En nuestro ejemplo, las aristas serían las carreteras que unen las ciudades.

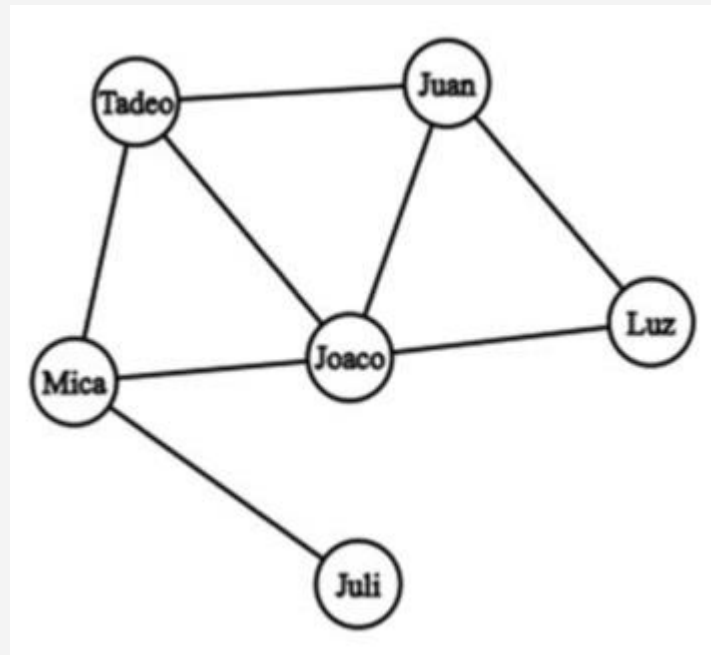


Tipos de grafos

Existen varios tipos de grafos, cada uno con propiedades distintas:

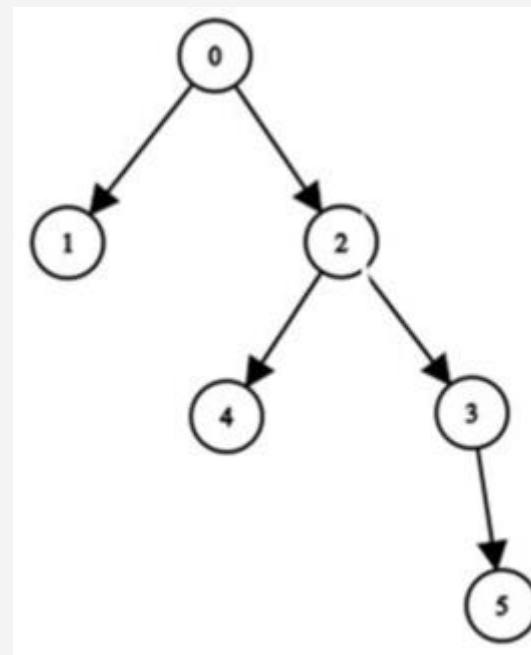
Grafo

No dirigido



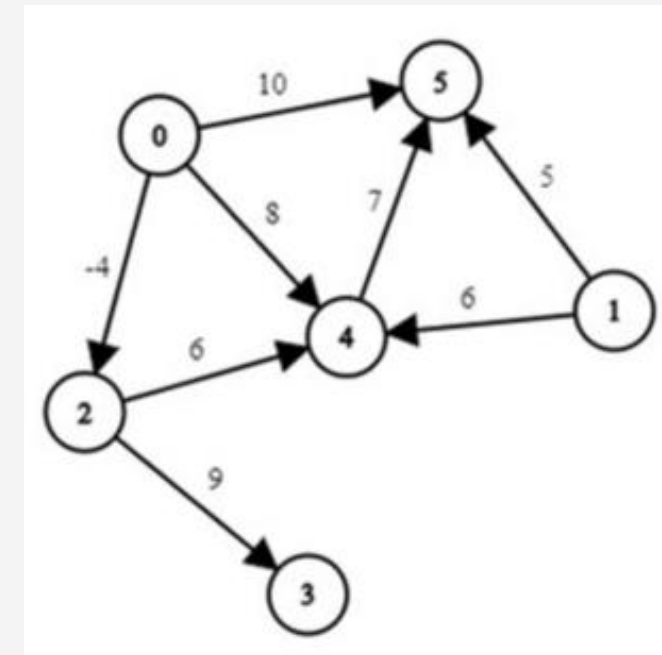
Grafo

Dirigido



Grafo

Ponderado

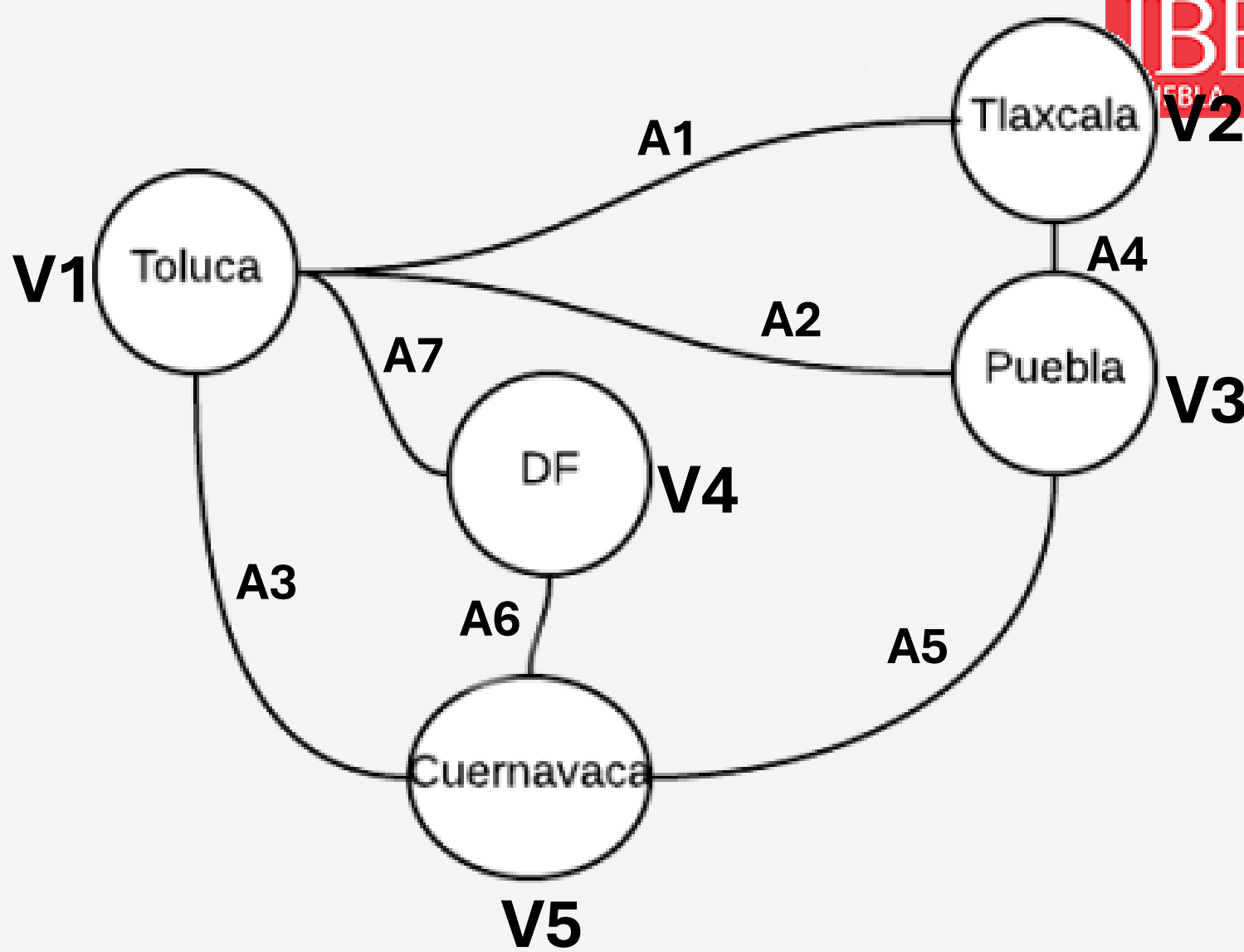


Tipos de grafos

Grafo no dirigido: Las aristas no tienen una dirección específica. Si la ciudad A está conectada a la ciudad B, la conexión es bidireccional, es decir, puedes ir de A a B y de B a A. Las aristas se representan como un par no ordenado de vértices (u, v).

Por ejemplo, según el siguiente grafo de ciudades tenemos:

- Vértices (Ciudades)**
- **V1:** Toluca
 - **V2:** Tlaxcala
 - **V3:** Puebla
 - **V4:** DF (Ciudad de México)
 - **V5:** Cuernavaca



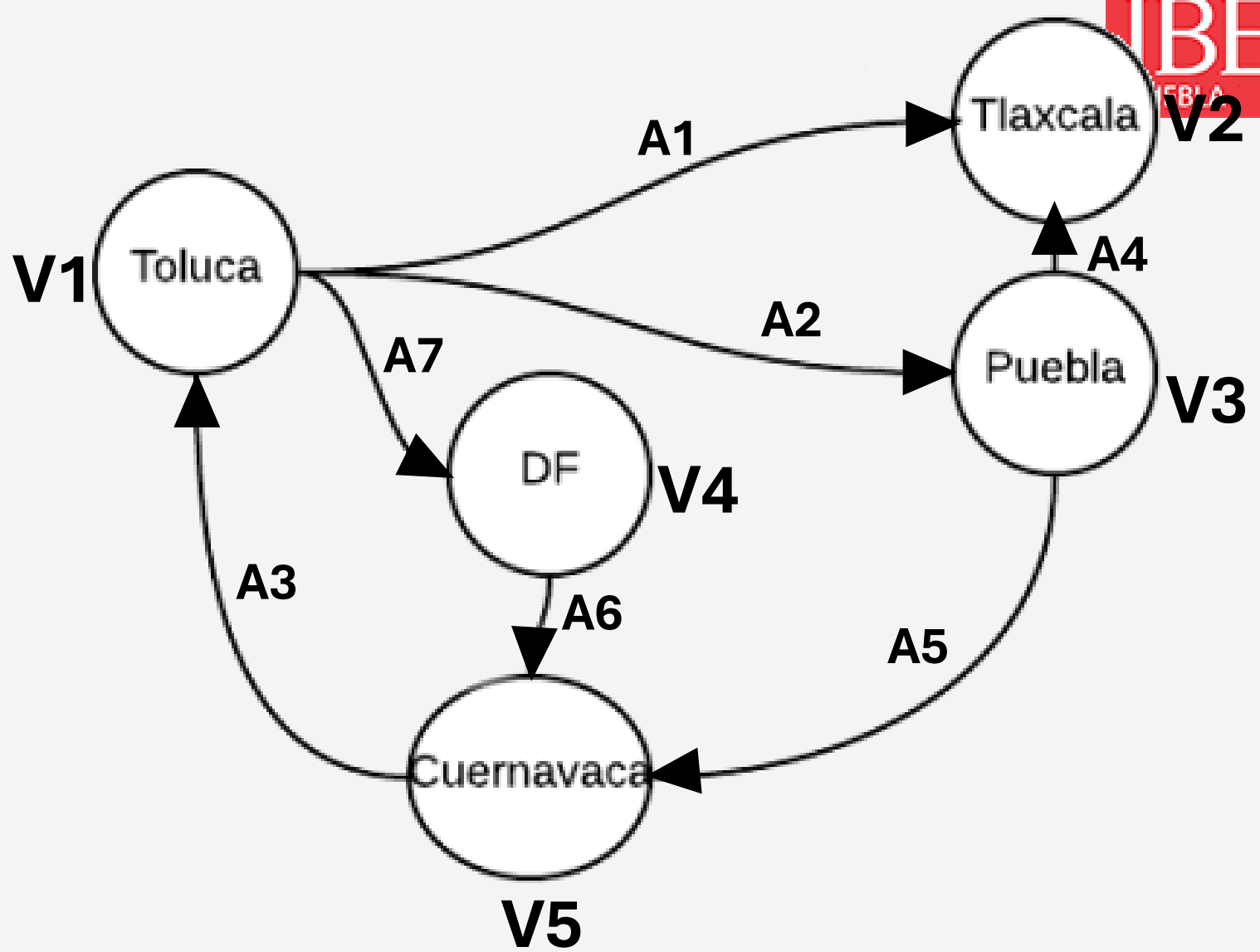
Nombre de la arista	Vértices conectados
A1	(Toluca, Tlaxcala)
A2	(Toluca, Puebla)
A3	(Toluca, Cuernavaca)
A4	(Tlaxcala, Puebla)
A5	(Puebla, Cuernavaca)
A6	(DF, Cuernavaca)
A7	(Toluca, DF)

Tipos de grafos

Grafo dirigido (dígrafo): Las aristas tienen una dirección. Por ejemplo, una calle de un solo sentido que te permite ir de A a B, pero no de B a A. Las aristas se representan como un par ordenado de vértices, por ejemplo, $\langle u,v \rangle$, lo que indica que la dirección va de u a v.

Por ejemplo, según el siguiente grafo de ciudades tenemos:

- Vértices (Ciudades)**
- **V1:** Toluca
 - **V2:** Tlaxcala
 - **V3:** Puebla
 - **V4:** DF (Ciudad de México)
 - **V5:** Cuernavaca



Nombre de la arista	Vértices conectados
A1	<Toluca, Tlaxcala>
A2	<Toluca, Puebla>
A3	<Cuernavaca, Toluca>
A4	<Puebla, Tlaxcala>
A5	<Puebla, Cuernavaca>
A6	<DF, Cuernavaca>
A7	<Toluca, DF>

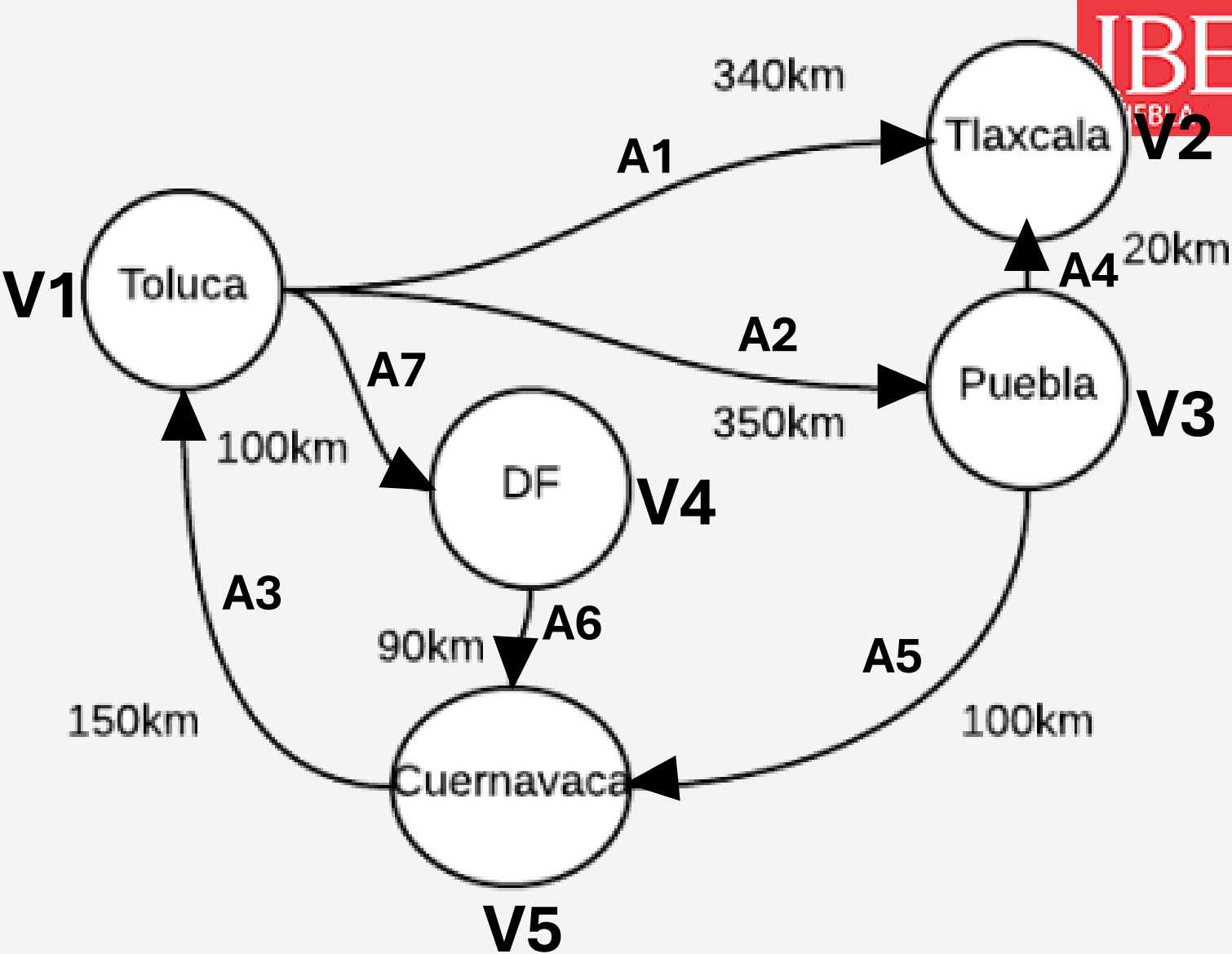
Tipos de grafos

Grafo ponderado: A cada arista se le asigna un valor numérico llamado peso. Este peso puede representar la distancia, el costo, el tiempo o cualquier otra medida asociada con la conexión. En el ejemplo de las ciudades, el peso podría ser la distancia en kilómetros entre ellas.

Por ejemplo, según el siguiente grafo de ciudades tenemos:

Vértices (Ciudades)

- **V1:** Toluca
- **V2:** Tlaxcala
- **V3:** Puebla
- **V4:** DF (Ciudad de México)
- **V5:** Cuernavaca



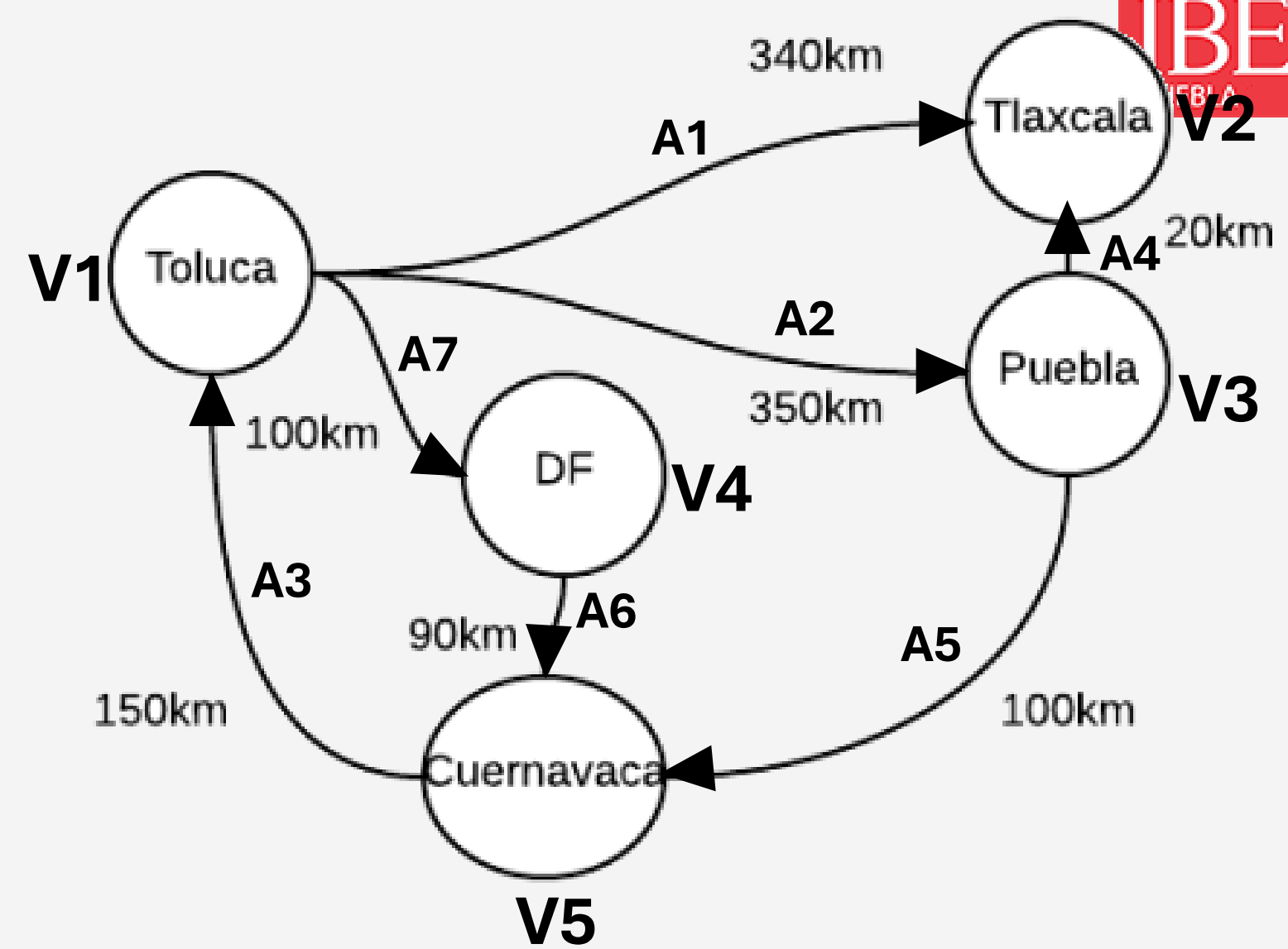
Nombre de la arista	Vértices conectados	Peso
A1	<Toluca, Tlaxcala>	340km
A2	<Toluca, Puebla>	350km
A3	<Cuernavaca, Toluca>	150km
A4	<Puebla, Tlaxcala>	20km
A5	<Puebla, Cuernavaca>	100km
A6	<DF, Cuernavaca>	90km
A7	<Toluca, DF>	100km

Matriz de adyacencia

La **matriz de adyacencia** (o matriz de conexión) es una forma de representar qué vértices están directamente conectados entre sí y, en el caso de un grafo ponderado, la distancia o costo de esa conexión.

Definición: Para un grafo con **N** vértices, la matriz es de dimensión **N×N**.

Regla: El elemento **A_{ij}** es el peso de la arista que va del vértice **V_i** al vértice **V_j**. Si no hay conexión directa, se usa un valor de 0 (cero) o ∞ (infinito, en el caso de grafos ponderados que miden costo/distancia).



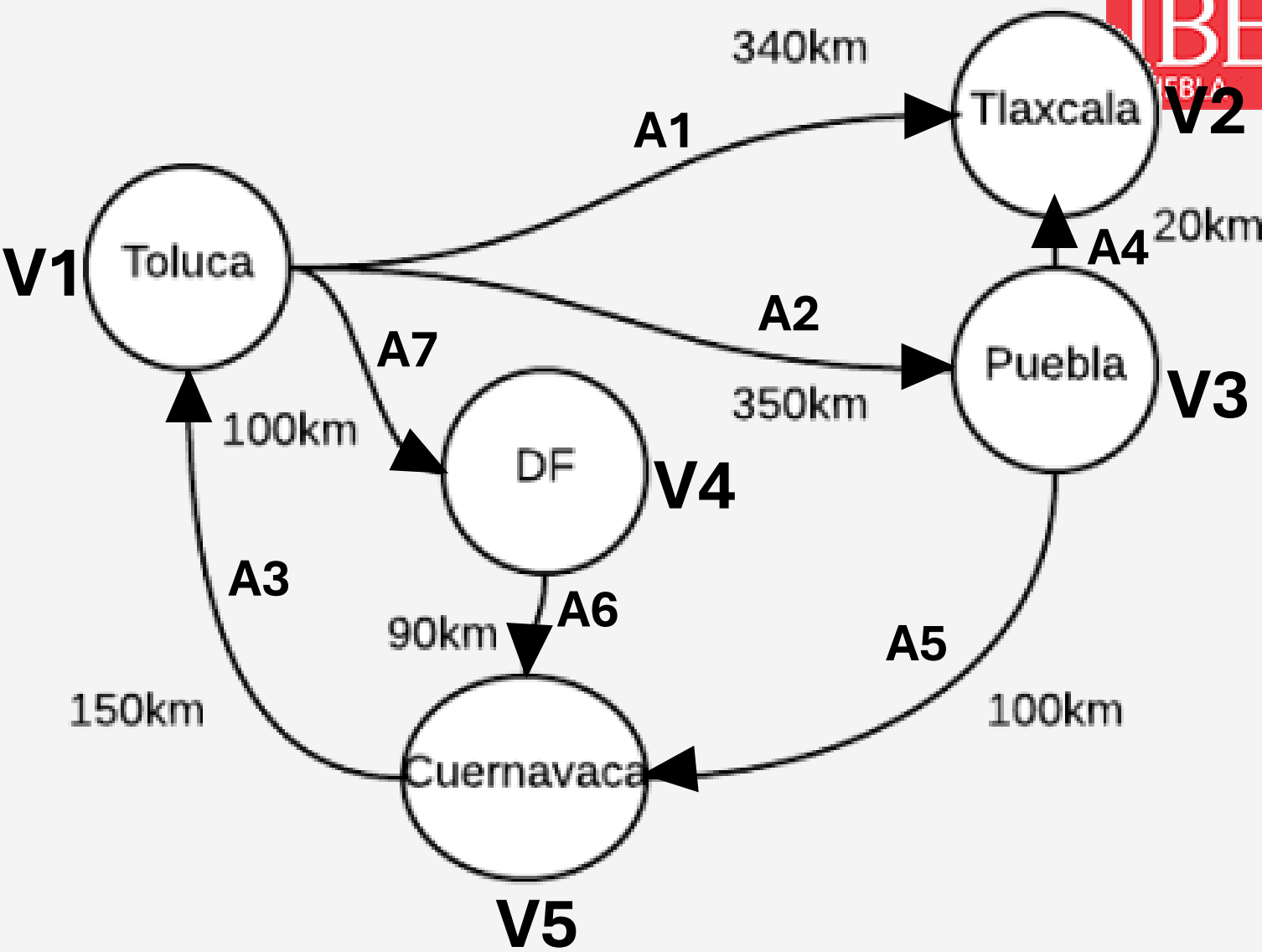
	V1	V2	V3	V4	V5
Origen ↓ / Destino →	(Toluca)	(Tlaxcala)	(Puebla)	(DF)	(Cuernavaca)
V1 (Toluca)	0	340 (A1)	350 (A2)	100 (A7)	0
V2 (Tlaxcala)	0	0	20 (A4)	0	0
V3 (Puebla)	0	0	0	0	100 (A5)
V4 (DF)	0	0	0	0	90 (A6)
V5 (Cuernavaca)	150 (A3)	0	0	0	0

Lista de adyacencia

A diferencia de la matriz de adyacencia, la lista de adyacencia no emplea una estructura NxN para representar qué vértices están directamente conectados entre sí.

La lista de adyacencia guarda, para cada vértice, un listado con los vértices a los que está conectado y, en grafos ponderados, también el peso de la arista.

Por ejemplo: Considerando el siguiente grafo de ciudades la lista de adyacencia sería:



Ciudad	Sin pesos	Con pesos
V1 (Toluca)	V1: [V2, V3, V4]	V1: [(V2, 340), (V3, 350), (V4, 100)]
V2 (Tlaxcala)	V2: []	V2: []
V3 (Puebla)	V3: [V2, V5]	V3: [(V2, 20), (V5, 100)]
V4 (DF)	V4: [V5]	V4: [(V5, 90)]
V5 (Cuernavaca)	V5: [V1]	V5: [(V1, 150)]

Matriz de Acceso (T)

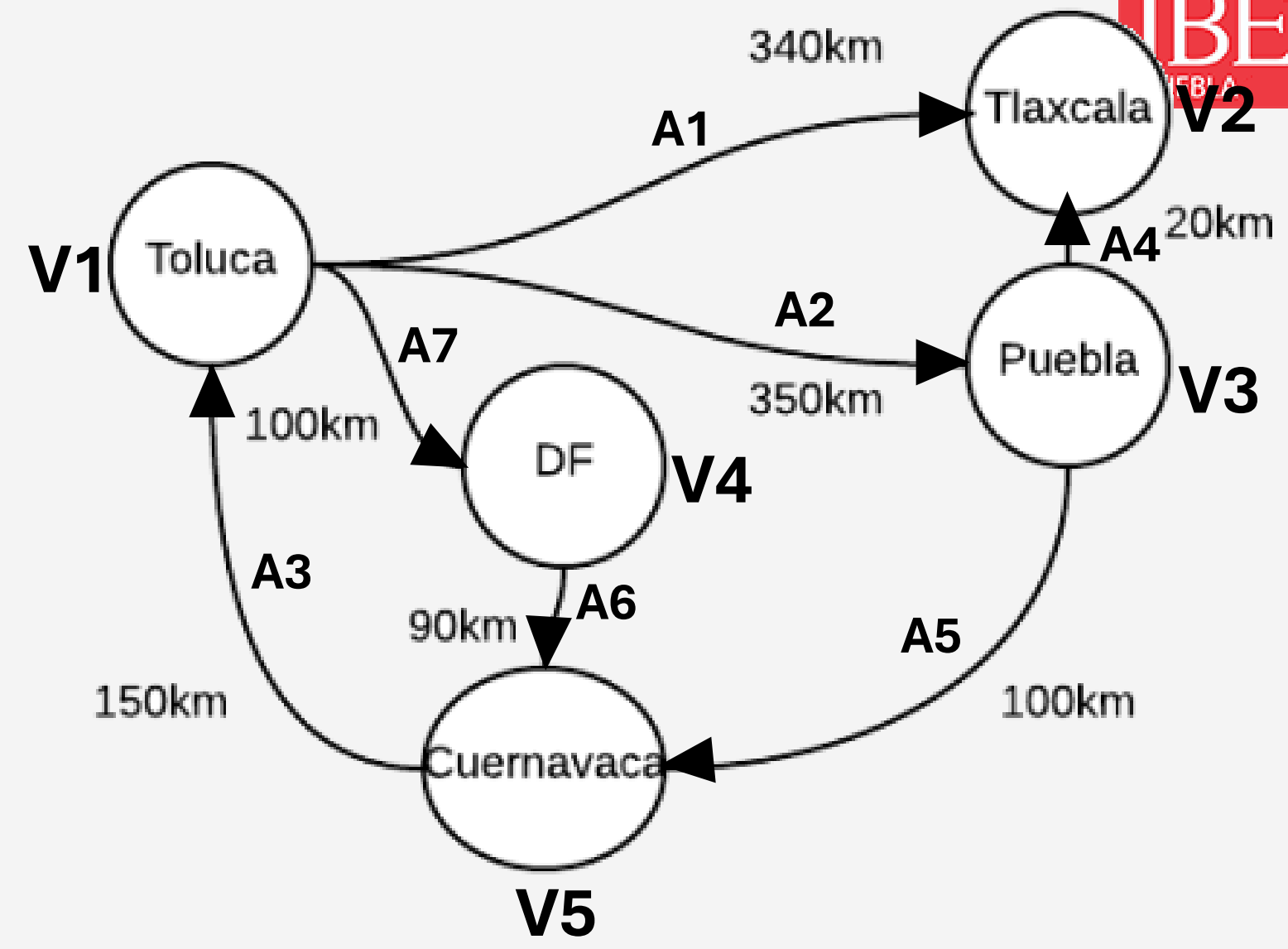
La **matriz de acceso** (o matriz de conectividad/alcanzabilidad) representa si existe un camino (en uno o más pasos) entre dos vértices. Ignora la distancia o peso, centrándose solo en la posibilidad de llegar.

Definición: Para un grafo con **N** vértices, la matriz es de dimensión **N×N**.

Regla: El elemento T_{ij} es:

- 1: si existe al menos un camino del vértice V_i al vértice V_j .
- 0: si no existe ningún camino.

Nota: Se considera que un vértice siempre es accesible a sí mismo (caminos de longitud cero), por lo que la diagonal principal siempre es 1.



	V1	V2	V3	V4	V5
Origen ↓ / Destino →	(Toluca)	(Tlaxcala)	(Puebla)	(DF)	(Cuernavaca)
V1 (Toluca)	1	1	1	1	1
V2 (Tlaxcala)	0	1	0	0	0
V3 (Puebla)	1	1	1	1	1
V4 (DF)	1	1	1	1	1
V5 (Cuernavaca)	1	1	1	1	1

Búsqueda en anchura (BFS)

La búsqueda en anchura, o **BFS** (Breadth-First Search) es una técnica de recorrido de grafos que busca determinar el alcance que tiene un nodo con respecto a todos los demás, explorando todos los vecinos en el nivel actual antes de pasar a los nodos del siguiente nivel y así sucesivamente

Su mayor ventaja es que garantiza encontrar el camino más corto en términos de número de pasos. Sin embargo, puede ser muy costoso en memoria. Es ideal para problemas donde la solución está relativamente cerca del inicio.

Características principales

- **Completo:** Siempre encuentra una solución si existe
- **Óptimo:** Encuentra la solución con el menor número de pasos (en grafos no ponderados)

Búsqueda en anchura (BFS)

Algoritmo:

I. **Inicialización.** Se requieren las siguientes estructuras de datos:

- a) **Lista de visitados:** Lista de los nodos alcanzados desde el nodo de referencia.
- b) **Lista de arcos:** Lista con los arcos necesarios para alcanzar los nodos.
- c) **Cola de gestión:** Lista necesaria para realizar el proceso de verificación de nodos visitados.

Se comienza con un nodo inicial, el cual se agrega a **visitados**.

II. **Proceso de Búsqueda:** El proceso principal se ejecuta en un bucle mientras la **cola** no esté vacía.

Paso 1. Se extrae el primer nodo de la cola y se agrega visitados. Este nodo es el que se va a explorar.

Paso 2. Se inspeccionan todos los vecinos del nodo extraído.

Paso 3. Por cada vecino:

- Se comprueba si ya ha sido visitado
- Si no ha sido visitado, se marca como visitado y se agrega al final de la cola.

III. **Finalización:** El algoritmo termina cuando la cola está vacía, es decir, cuando se ha encontrado el nodo especificado.

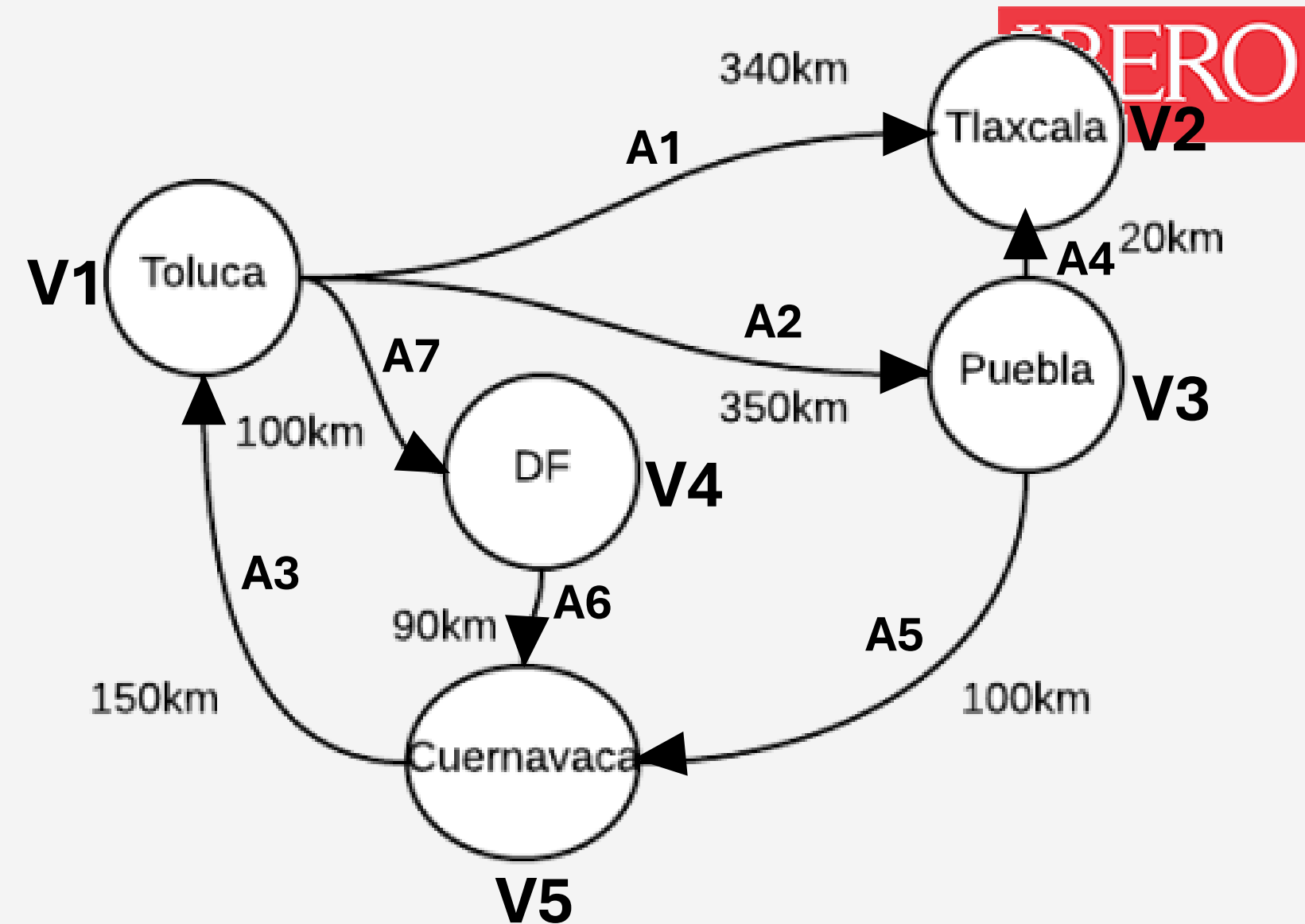
Ejemplo: Ciudades de México

Instrucciones: Considerando el siguiente grafo aplica el algoritmo BFS, tomando como punto de partida ciudad de Toluca.

visitados=

arcos=

cola=



Búsqueda en profundidad (DFS)

La búsqueda en profundidad, o **DFS** (Deep-First Search) es un algoritmo de recorrido de árboles o grafos que explora tan lejos como es posible a lo largo de cada rama antes de retroceder. Imagina que estás en un laberinto. El DFS te dice que vayas por un camino hasta el final y, si no es el correcto, regreses al último cruce y tomes otra dirección.

Es conocido por su eficiencia de memoria al no tener que almacenar todos los nodos de un mismo nivel, a diferencia de otros algoritmos como BFS. Además, es muy efectivo para detectar ciclos en grafos.

Características principales

- **Recorrido corto no garantizado:** No garantiza que encontrará la ruta más corta entre dos nodos.
- **Backtraking:** Al emplear una estructura de pila, cuando DFS llega a un nodo sin vecinos no visitados, el algoritmo retrocede (hace backtracking) al nodo anterior y continúa la exploración desde allí.

Búsqueda en profundidad (DFS)

Algoritmo:

I. Inicialización

- Elige un nodo de inicio y crea una pila (una estructura de datos de "último en entrar, primero en salir" o LIFO) para almacenar los nodos a visitar.
- Crea un conjunto o lista para registrar los nodos que ya has visitado. Esto evita que entres en un ciclo infinito.

II. Empujar el nodo inicial

- Empuja el nodo de inicio a la pila.

III. Bucle de recorrido

- Mientras la pila no esté vacía, repite los siguientes pasos:
 - **Paso 1.** Saca el nodo superior de la pila. Este es tu nodo actual.
 - **Paso 2.** Si el nodo actual no ha sido visitado:
 - Marca el nodo actual como visitado.
 - Procesa el nodo actual (por ejemplo, imprímelo o guárdalo como parte de la solución).
 - **Paso 3.** Para cada uno de los vecinos no visitados del nodo actual:
 - Empuja al vecino a la pila.

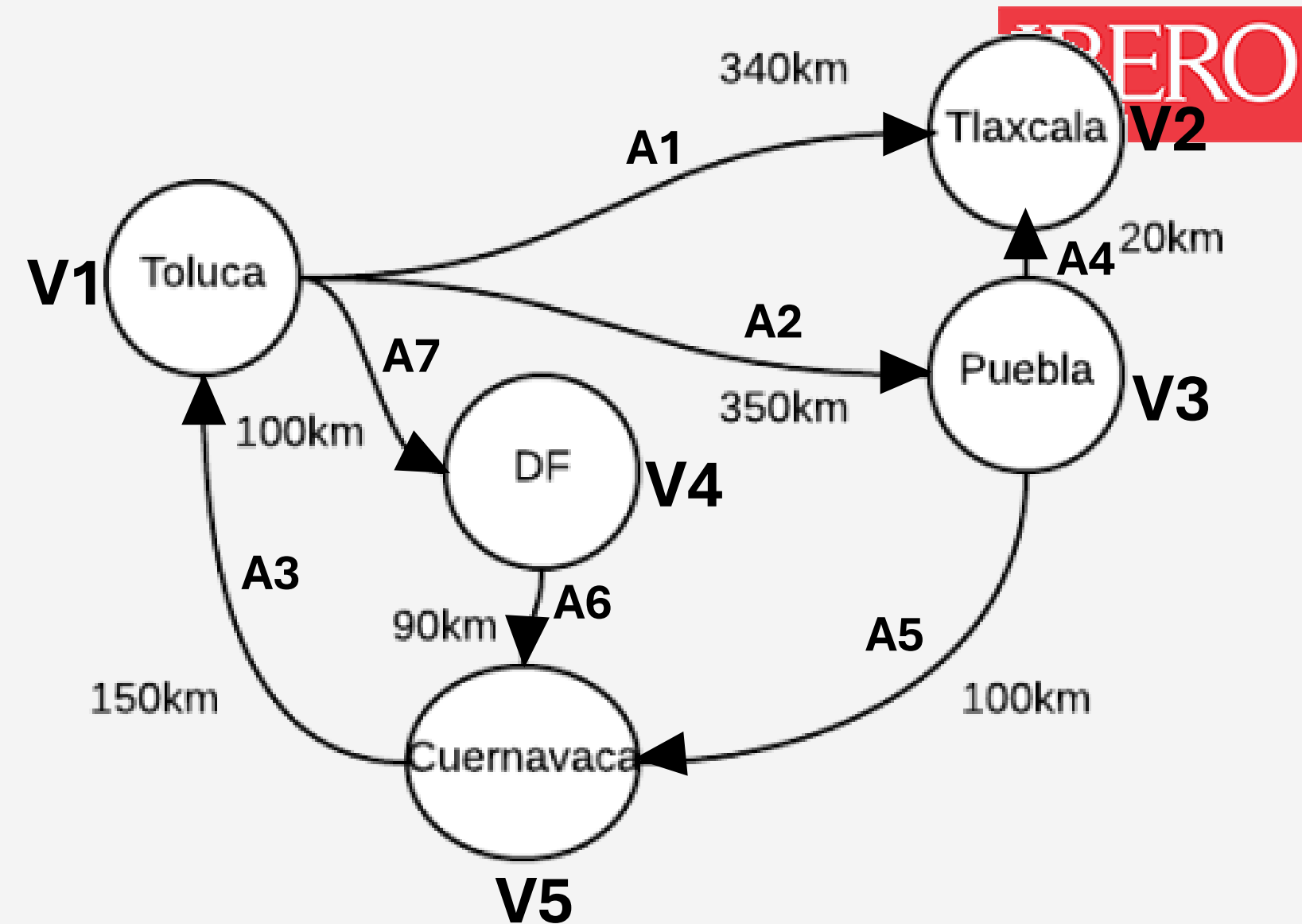
Ejemplo: Ciudades de México

Instrucciones: Considerando el siguiente grafo aplica el algoritmo DFS, tomando como punto de partida ciudad de Puebla.

visitados=

arcos=

pila=



¡gracias!

IBERO
PUEBLA